Hadoop & Spark - ISEN 2025/2026

Lab: Transformations & Aggregations

For this Lab you will need:

- The 3.6 Lab

As a reminder

Check 3.1 to login in NGROK, Kaggle, install Java or Spark and import libraries.

Transformations

Try some transformations on the Yellow Taxi Dataset on Kaggle: https://www.kaggle.com/datasets/microize/newyork-yellow-taxi-trip-data-2020-2019

You can refer to the previous exercise.

Let's make sure we start from the same data by selecting one month (March 2020 for example).

Transformations

```
yellowTaxiSchema = (
                        StructType
                        ([
                            StructField("VendorId"                , IntegerType()   , True),
                            StructField("lpep_pickup_datetime"    , TimestampType() , True),
                            StructField("lpep_dropoff_datetime"   , TimestampType() , True),
                            StructField("passenger_count"         , DoubleType()    , True),
                            StructField("trip_distance"           , DoubleType()    , True),
                            StructField("RatecodeID"              , DoubleType()    , True),
                            StructField("store_and_fwd_flag"      , StringType()    , True),
                            StructField("PULocationID"            , IntegerType()   , True),
                            StructField("DOLocationID"            , IntegerType()   , True),
                            StructField("payment_type"            , IntegerType()   , True),
                            StructField("fare_amount"             , DoubleType()    , True),
                            StructField("extra"                   , DoubleType()    , True),
                            StructField("mta_tax"                 , DoubleType()    , True),
                            StructField("tip_amount"              , DoubleType()    , True),
                            StructField("tolls_amount"            , DoubleType()    , True),
                            StructField("improvement_surcharge"   , DoubleType()    , True),
                            StructField("total_amount"            , DoubleType()    , True),
                            StructField("congestion_surcharge"    , DoubleType()    , True),
                        ])
                    )
```

Transformations

Declare map of default values:

```
defaultValueMap = {'payment_type': 5, 'RateCodeID': 1}
```

Read file

```
yellowTaxiDF = (
                  spark
                    .read
                    .option("header", "true")
                    .schema(yellowTaxiSchema)
                    .csv("yellow_tripdata_2020-01.csv")
                )
```

Transformations

Cleanup data by applying data quality checks

```
yellowTaxiDF = (
                  yellowTaxiDF
                      .where("lpep_pickup_datetime >= '2020-01-03' AND lpep_dropoff_datetime < '2020-01-04'")
                      .where("passenger_count > 0")
                      .filter(col("trip_distance") > 0.0)
                      .na.drop('all')
                      .na.fill(defaultValueMap)
                      .dropDuplicates()
               )
```

```
print("Rows = " + str( yellowTaxiDF.count()) )
```

Check if everything is ok

```
yellowTaxiDF.printSchema()
```

Transformations examples

Here are some examples of transformations you can apply. We do not use any action on purpose to have the cells executing quickly.

You can select from multiple syntaxes:

```
yellowTaxiDF = (
                    yellowTaxiDF
                         # Select only limited columns
                         .select(
                                   "VendorID", # syntax 1
                                   col("passenger_count").cast(IntegerType()), # syntax 2
                                   column("trip_distance").alias("TripDistance"), # syntax 3
                                   yellowTaxiDF.lpep_pickup_datetime, # syntax 4
                                   "lpep_dropoff_datetime",
                                   "PUlocationID",
                                   "DOlocationID",
                                   "RatecodeID",
                                   "total_amount",
                                   "payment_type",
                                 )
                )

yellowTaxiDF.printSchema()
```

Transformations examples

We can rename columns:

```
yellowTaxiDF = (
                    yellowTaxiDF
                         .withColumnRenamed("passenger_count", "PassengerCount")
                         .withColumnRenamed("lpep_pickup_datetime", "PickupTime")
                         .withColumnRenamed("lpep_dropoff_datetime", "DropTime")
                         .withColumnRenamed("PUlocationID", "PickupLocationId")
                         .withColumnRenamed("DOlocationID", "DropLocationId")
                         .withColumnRenamed("total_amount", "TotalAmount")
                         .withColumnRenamed("payment_type", "PaymentType")
                )

yellowTaxiDF.printSchema()
```

Transformations examples

We can add new columns based on already existing ones. Lets create three columns `TripYear`, `TripMonth` and `TripDay`:

- expr doc
- dayofmonth doc

```
yellowTaxiDF = (
                  yellowTaxiDF

                        .withColumn("TripYear", year(col("PickupTime"))) # use the year function

                        .select(
                                    "*",

                                    expr("month(PickupTime) AS TripMonth"), # use expression

                                    dayofmonth(col("PickupTime")).alias("TripDay")
                              )
                )

yellowTaxiDF.printSchema()
```

Transformations examples

Lets calculate the time in minutes of every trip :

```
yellowTaxiDF = (
                  yellowTaxiDF
                        .withColumn("TripTimeInMinutes",
                                        round(
                                                (unix_timestamp(col("DropTime"))
                                                     - unix_timestamp(col("PickupTime")))

                                                  / 60
                                            )
                                      )
                )

yellowTaxiDF.printSchema()
```

Transformations examples

This formulation is a bit complex, let's use a dynamic expression:

```
# you can declare the operation beforehand:
tripTimeInSecondsExpr = unix_timestamp(col("DropTime")) - unix_timestamp(col("PickupTime"))

tripTimeInMinutesExpr = round(tripTimeInSecondsExpr / 60)
```

```
yellowTaxiDF = (
                  yellowTaxiDF
                        .withColumn("TripTimeInMinutes", tripTimeInMinutesExpr)
                )
```

```
yellowTaxiDF.printSchema()
```

Transformations examples

Let's created a derived column trip type: (cf the dataset doc)

```
tripTypeColumn = (
                    when(
                            col("RatecodeID") == 6, # 6 -> Group ride
                              "SharedTrip"
                         )
                    .otherwise("SoloTrip")
                 )


yellowTaxiDF = (
                  yellowTaxiDF

                       .withColumn("TripType", tripTypeColumn)

                       .drop("RatecodeID")
               )

yellowTaxiDF.printSchema()
```

Aggregations

Here are some aggregations commands:

```
yellowTaxiReport = (
    yellowTaxiDF
      .groupBy("PickupLocationId")
      .agg(
          avg("TripTimeInMinutes").alias("AvgTripDuration"),
          sum("TotalAmount").alias("SumAmount")
      )
      .orderBy(col("PickupLocationId").desc())
)
yellowTaxiReport.show()
```

See the plan

Note that we did not execute any operation in the previous cell. You can see the unexecuted plan with `explain`:
(see doc)

```
yellowTaxiDF.explain("simple")
```

```
yellowTaxiDF.explain("codegen")
```

```
yellowTaxiDF.explain("cost")
```

```
yellowTaxiDF.explain("formatted")
```

```
yellowTaxiDF.explain("extended")
```

Let's execute all this:

```
yellowTaxiDF.show()
```

Exercise 1: Covid

For this exercise let's use the COVID-19 dataset from Kaggle (sudalairajkumar/novel-corona-virus-2019-dataset).

1- First, download the dataset and load it into a PySpark DataFrame (Declare your Schema)
2- Perform some transformations on the data:

1. Convert the `ObservationDate` and `Last_Update` columns to TimestampType.
2. Rename the `Province_State` and `Country_Region` columns to `Province` and `Country` respectively.
3. Create a new column `Active` that represents the number of active cases (`Confirmed - Deaths - Recovered`).

3- Perform some aggregations on the data:

1. Calculate the total number of confirmed, deaths, recovered, and active cases per country.
2. Calculate the average number of confirmed cases per day globally.
3. Find the top 5 countries with the highest number of confirmed cases.